T2 AI Route Planner

Отчёт о реализации проекта

Платформа оптимизации логистических маршрутов для торговых представителей Республики Мордовия

Backend: FastAPI · PostgreSQL · SQLAlchemy async · Alembic Frontend: Vue 3 · TypeScript · TailwindCSS
AI: Qwen 0.5B · Llama 3.2 1B (GGUF)

1. Обзор проекта

T2 AI Route Planner — платформа для планирования и оптимизации маршрутов торговых представителей по торговым точкам (ТТ) Республики Мордовия. Система автоматически строит месячные расписания визитов с учётом категорий ТТ, форс-мажоров и праздников.

Ключевые параметры

ТТ в базе	250 торговых точек, 23 района Мордовии
Категории ТТ	A (20%) — 3 визита/мес, B (30%) — 2/мес, C (20%) — 1/мес, D (30%) — 1/квартал
Макс. ТТ/день	14 (540 мин − 30 мин обед) / 35 мин на ТТ
Рабочее время	09:00–18:00 (пн–пт), учёт 17 праздников 2026
Версия	1.3.0 (март 2026)

Технологический стек

Backend	FastAPI (Python 3.11), SQLAlchemy 2.0 async, Alembic, PostgreSQL 15
Frontend	Vue 3 + TypeScript, TailwindCSS, Vite, Axios
AI / LLM	Qwen 0.5B-Instruct Q4_K_M (~400 MB), Llama 3.2 1B Q4_K_M (резерв)
DevOps	Docker Compose, Nginx, GitHub Actions CI/CD
Экспорт	openpyxl (5 листов Excel), fpdf2 (PDF-отчёт)

2. Схема базы данных (10 таблиц)

locations	TT: id, name, lat, lon, category, city, district, address, time_window_*
sales_reps	Сотрудники: id, name, status (active/sick/vacation/unavailable), created_at
visit_schedule	Плановые визиты: id, location_id, rep_id, planned_date, status, created_at
visit_log	Фактические визиты: id, schedule_id, location_id, rep_id, visited_date, time_in, time_out, notes
force_majeure_events	Форс-мажоры: id, type, rep_id, event_date, description, affected_tt_ids (JSON), redistributed_to (JSON)
holidays	Праздники: date (PK), name, is_working (bool) — управляемые через API
audit_log	Журнал изменений: id, action, table_name, record_id, old_value, new_value, details, created_at
routes	Маршруты LLM: id, name, locations_order (JSON), total_distance, total_time, total_cost, model_used
metrics	Метрики LLM: id, route_id, model_name, response_time_ms, quality_score, cost
optimization_results	Результаты оптимизации: id, original_route, optimized_route, improvement_pct, model_used

3. API Endpoints (32 эндпоинта)

Система

- GET /health — Health check (DB + LLM + disk_free_mb + visits_today)
- GET /api/v1/health — то же для фронтенда

Локации

- GET/POST /api/v1/locations/
- GET/PUT/DELETE /api/v1/locations/{id}
- POST /api/v1/locations/upload — загрузка XLSX/CSV/JSON

Оптимизация

- POST /api/v1/optimize — Greedy nearest-neighbor
- POST /api/v1/qwen/optimize, /api/v1/llama/optimize
- POST /api/v1/optimize/variants — 3 варианта + pros/cons
- POST /api/v1/optimize/confirm — подтверждение варианта

Сотрудники

- GET/POST /api/v1/reps/
- PATCH/DELETE /api/v1/reps/{id}

Расписание

- POST /api/v1/schedule/generate?force=true|false — генерация плана
- GET /api/v1/schedule/?from_date=&to_date=
- PATCH /api/v1/schedule/{id} — смена статуса (машина состояний)

Визиты

- GET/POST /api/v1/visits/
- GET /api/v1/visits/stats

Форс-мажоры

- POST /api/v1/force_majeure/ — регистрация + автоперераспределение
- GET /api/v1/force_majeure/

Праздники

- GET /api/v1/holidays/?month=YYYY-MM или ?year=YYYY
- PATCH /api/v1/holidays/{date} — is_working=true|false, возвращает affected_visits_count

Экспорт

- GET /api/v1/export/schedule?month= — Excel 5 листов
- POST /api/v1/import/schedule — загрузка заполненного Excel

Аналитика

- GET /api/v1/insights?month=YYYY-MM
- GET /api/v1/metrics/
- GET /api/v1/routes/, /api/v1/routes/{id}

4. Алгоритм планировщика (SchedulePlanner)

Логика планирования

- Загрузка нерабочих праздников из БД (таблица holidays, is_working=False) за целевой месяц
- _is_working_day(d, non_working) — проверяет weekday < 5 и d not in non_working
- Сортировка ТТ по (целевая дата, приоритет категории): A→B→C→D
- Распределение: A = 1-я/2-я/3-я рабочие недели, B = 1-я/3-я, C = середина, D = 1-й рабочий день квартала
- Round-robin по сотрудникам: max(rep_day_slots) → наиболее свободный
- Lookahead до 30+ дней вперёд при переполнении дня

Учёт праздников

- HOLIDAYS_2026 — 17 нерабочих дней (1–9 янв, 23 фев, 8–9 мар, 1 мая, 9–11 мая, 12 июн, 4 ноя)
- Сидируются в БД при первом старте приложения
- Пользователь может изменить статус любого дня через PATCH /holidays/{date}
- При переводе дня в нерабочий API возвращает affected_visits_count — сколько визитов затронуто
- В модалке генерации плана — список праздников месяца с чекбоксами (по умолчанию нерабочие)
- Кнопка «Праздники» в ScheduleView — управление всеми днями года

Торговый представитель

planned	→ completed, skipped, cancelled, rescheduled
skipped	→ planned, cancelled (+ авто-создание rescheduled-визита)
rescheduled	→ completed, skipped, cancelled
completed	терминальный
cancelled	терминальный

Форс-мажор

- Типы: illness, weather, vehicle_breakdown, other
- При создании: rep.status = sick, planned-визиты → rescheduled, round-robin на активных
- Ограничение: MAX_TT_PER_DAY = 14 на сотрудника в день

5. История версий

Фаза 1–2: Базовый backend + LLM

- FastAPI + SQLAlchemy + PostgreSQL
- QwenClient + LlamaClient (GGUF)
- Vue 3 frontend с базовым layout
- Docker Compose (5 сервисов: postgres, redis, backend, frontend, nginx)

Фаза 3: MVP Production-Ready

- CRUD локаций (CSV/JSON/XLSX upload)
- Dashboard: 3 LLM-стратегии, Analytics, Benchmark
- Деплой на сервер, GitHub Actions CI/CD
- Nginx SPA routing + API reverse proxy

Фаза 4 — v1.2.0: Excel + Расписание

- 4 новые таблицы: SalesRep, VisitSchedule, VisitLog, ForceMajeureEvent
- SchedulePlanner: алгоритм A/B/C/D, MAX 14/день
- ForceMajeureService с round-robin перераспределением
- Excel экспорт (4 листа) + импорт + AuditLog
- State machine визитов (422 при invalid transition)
- Дублирование плана: POST /schedule/generate?force=true
- Frontend: ScheduleView + RepsView + модалка выбора LLM-варианта
- 91 backend-тестов, 190 frontend-тестов

Фаза 5 — v1.3.0: Управление праздниками

- Таблица holidays (date PK, name, is_working=False)
- Сидирование 17 праздников 2026 при старте
- GET /holidays/?month= или ?year= — список праздников
- PATCH /holidays/{date} — смена статуса + affected_visits_count
- _is_working_day() в SchedulePlanner — учёт праздников при генерации
- ScheduleView: кнопка «Праздники», чекбоксы в модалке генерации
- Excel: 5-й лист «Журнал изменений» (AuditLog)

6. Тестирование и покрытие (v1.3.0)

Статистика

Backend (pytest)	91 тест, ~64% coverage
Frontend (vitest)	190 тестов, ~70% coverage
ML / Benchmark	15 тестов, ~80% coverage
TypeScript	0 ошибок (vue-tsc --noEmit)
Итого	296+ тестов

Ключевые тест-кейсы

- POST /optimize — < 100 мс, 250 локаций
- POST /optimize/variants — 3 варианта за 30–90 сек (LLM)
- POST /schedule/generate — < 500 мс, 250 TT, 4 сотрудника
- PATCH /schedule/{id} status=skipped → авто-создание rescheduled
- POST /force_majeure — перераспределение < 200 мс
- GET /export/schedule — Excel 5 листов > 5 KB
- GET/PATCH /holidays/ — управление праздниками с affected_visits_count

CI/CD (GitHub Actions)

- flake8 линтер на каждый push
- pytest + vitest в pipeline при pull request
- Автодеплой на сервер 100.120.184.98 (Tailscale VPN)

7. Инфраструктура и деплой

Сервер	Ubuntu 24.04, ~55 GB disk, Tailscale VPN
Frontend	http://100.120.184.98 (Nginx)
API	http://100.120.184.98:8000
Swagger UI	http://100.120.184.98:8000/docs
PostgreSQL	100.120.184.98:5432
GitHub	https://github.com/JellyfishKa/T2_project

- Docker Compose: postgres, redis, backend, frontend, nginx
- QWEN_MODEL_ID = qwen2-0_5b-instruct-q4_k_m.gguf (~400 MB, 0.6 GB RAM)
- Llama 3.2 1B — резерв (1.2 GB RAM, OOM-риск на 4 GB VM, отключён по умолчанию)
- Redis 7 — кэш (используется в middleware)
- Alembic migrations: 001_initial → 002_add_reps_schedule → 003_add_audit_log → (holidays via CREATE TABLE IF NOT EXISTS)